

0.48. Les réglages sous un nom : Carre

Carre ronde = { ... }; range les sept réglages sous un nom, en haut du programme ; carre(ronde); s'en sert.

1. Ce qu'il faut comprendre

Sept nombres à la suite se lisent mal : lequel est la taille, lequel la vitesse ? On les range donc **sous un nom**, en haut du programme, comme `Mot` au chapitre 1.

Ce qui est nouveau ici : le type `Carre`. `Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 2 };` range les sept réglages, **dans le même ordre** que `carre(...)`, sous le nom `ronde`.

`carre(ronde);` fait le carré avec ces réglages. Le compilateur remplace `carre(ronde)` par `carre(10, 8, ALPHABET[0], 2, 1, 250, 2)` : rien n'est rangé en mémoire, c'est gratuit.

L'avantage : pour changer le carré, on ne touche qu'à la ligne du haut, où chaque réglage a son titre au-dessus. Le programme, lui, ne dit plus que « fais le carré ronde ».

Il faut les sept réglages, ni plus ni moins. Il en manque un ? Le compilateur le dit, et donne un exemple complet.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Carre>` → [0.47.3. Carre — les réglages d'un carré sous un nom](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <carre>` → [0.33.3. La fonction carre\(\) — tourner en carré](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
carre()         440 o ← main() — une fonction fournie par le langage
main()          38 o ← démarrage — ta fonction
VBlank          8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage   28 o ← image() — attend l'image suivante
AttendreVBlank  16 o ← démarrage — attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() — efface des cases
CopierTuiles    18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage — vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```

poser()   × 4    52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3    45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()   × 4    12 o sur place, dans carre(), main() + AttendreImage (28 o)
Tes fonctions appelées :
carre()   × 1    31 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// Carre : un type pour ranger les sept réglages d'un carré sous UN nom,
// comme Mot range la place et le texte d'un mot (chapitre 1).
//
// Carre  ronde  = { 10, 8, ALPHABET[0], 2, 1, 250, 2 };
// |      |      |
// |      |      +-- les sept réglages, DANS LE MÊME ORDRE que carre(...),
// |      |      entre accolades { }, séparés par des virgules
// |      +----- le nom qu'on choisit : ronde
// +----- le type : Carre (avec une majuscule)
//
//          x  y  tuile      taille  sens  vitesse  tours

#include <Carre>      // les sept réglages d'un carré sous un seul nom
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <carre>      // une tuile qui tourne en carré

```

2. UNE VARIABLE — RONDE

```

Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 2 };
// autour de (10, 8), la lettre A, 5 × 5, sens des aiguilles, 250 ms par pas, 2 tours

```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    // carre(ronde) veut dire : « fais le carré avec les réglages de ronde ».
    // Le compilateur la remplace par carre(10, 8, ALPHABET[0], 2, 1, 250, 2) :
    // c'est exactement le même programme, mais plus facile à lire.
    carre(ronde);

    // Pour changer le carré, on ne touche QUE la ligne de « ronde », en haut :
    // chaque réglage a son titre juste au-dessus.
    while (true) {
        image();           // attend l'image suivante (60 par seconde)
    }
}

```

3. Ce qui se passe, étape par étape

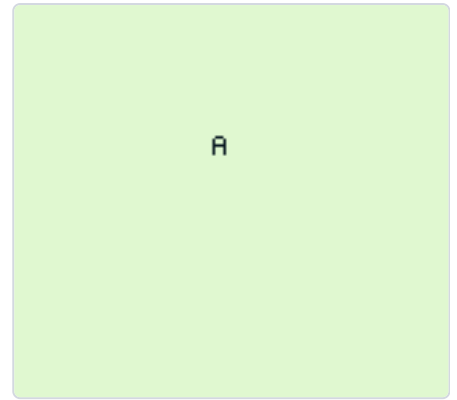
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



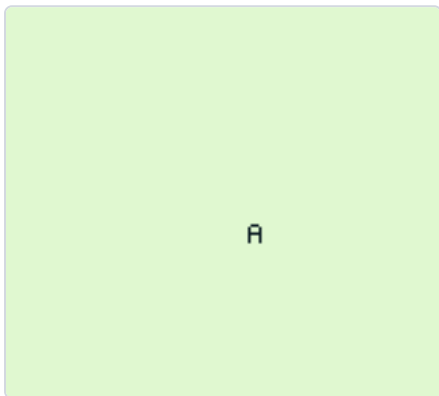
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui fait deux tours de 5×5 autour de (10, 8), et revient au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ deux tours de 5×5 : deux passages au coin (12, 6) (2)

✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.48.1. Les réglages sous un nom : Carre — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++